

Na temat argumentów funkcji

ZADANIE

Zaczniemy od bardzo prostego kodu, proszę napisać definicję funkcji `def fib(n)`, zwracającej listę kolejnych wyrazów ciągu Fibonacciego. Uwaga – najpierw **wersja iteracyjna**. Koszt obliczeniowy to $O(n)$

```
def fib(n):  
    result = []  
    # tu kod  
    return result
```

Wyraz n:	1, 2, 3, 4...
Element ciągu:	0, 1, 1, 2...

Następnie sprawdzić (nadal wersja iteracyjna), co się stanie jeśli **zamiast** wewnętrznej listy wymyślibyśmy coś takiego:

```
def fib(n, result = []):  
    # tu kod  
    return result
```

Wywołajmy kilka razy. Jak takie coś „naprawić”?

 ZADANIE**Wersja iteracyjna:**

```
def fib(n):
    result = [0, 1]
    for _ in range(2, n):
        result.append(result[-1] + result[-2])
    return result[:n]
```

Oczywiście wariant liczący
tylko n-ty wyraz ciągu:

```
def fib(n):
    if n < 2:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Problem – parametr domyślny jest tworzony jednorazowo w chwili definiowania funkcji, potem jest współdzielony:

```
def fib(n, result = []):
    if n == 1:
        result.extend([0])
        return result
    result.extend([0, 1])
    for _ in range(2, n):
        result.append(result[-1] + result[-2])
    return result
```

Efekt: każde kolejne wywołanie funkcji, bez podania
zewnętrznego argumentu, używa tego samego obiektu listy.
W implementacji jak po lewej, powoduje to efekt dodawania
kolejnych wyników do tej samej listy:

```
>>> print(fib(3))
[0, 1, 1]
>>> print(fib(5))
[0, 1, 1, 0, 1, 1, 2, 3]
```

Jeden ze sposobów naprawienia:

```
if n == 1:
    result[:] = [0] # nadpisz zawartość, NIE zmieniaj referencji
    return result
result[:] = [0, 1] # nadpisz zawartość
```

Inny sposób naprawienia:

```
def fib(n, result=None):
    if result is None: # utwórz nową listę przy każdym wywołaniu
        result = [0] if n == 1 else [0, 1]
    for _ in range(2, n):
        result.append(result[-1] + result[-2])
    return result[:n] # również return result - zależy czy ktoś poda argument
```

Ciekawostka: do obiektów argumentów domyślnych można się dostać poprzez atrybut `__defaults__`,

np. przed wywołaniem fib mamy:

```
>>> print(fib.__defaults__)
([],)
```

po wywołaniu fib(3) mamy:

```
>>> print(fib.__defaults__)
([0, 1, 1],)
```

można zresetować:

```
fib.__defaults__[0].clear()
lub nadpisać: fib.__defaults__ = ([],)
```

 ZADANIE

Napisać wersję rekurencyjną. Koszt to $O(2^n)$.

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

Taka wersja zwraca wartość n-tego elementu ciągu Fibonacciego, zatem nie listę z wyrazami od pierwszego do n-tego. Jak łatwo sprawdzić, jest to bardzo niewydajny sposób liczenia, za każdym razem liczy od początku.

Poprawiając koszt proszę zdefiniować zagnieżdżoną definicję pomocniczej funkcji, która sprawdzi w słowniku, czy wartość już nie była wcześniej policzona i tylko nowe wartości będzie wyliczać.

```
def fib(n):
    # początkowe wartości w słowniku
    def helper(k):
        # kod zwracający pozycję słownika
    return helper(n)
```

```
def fib(n):
    memo = {0: 0, 1: 1} # słownik
    def helper(k):
        if k in memo:
            return memo[k]
        memo[k] = helper(k - 1) + helper(k - 2)
        return memo[k]
    return helper(n)
```

Użycie zagnieżdżonej funkcji wynika z faktu, że „podręczna pamięć” (słownik) musi **przetrwąć między wywołaniami rekurencyjnymi**. Dlatego musi być „na zewnątrz” *pojedynczego* wywołania. Można to zrealizować dodając argument:

```
def fib(n, memo={0:0,1:1}):
    if n in memo: return memo[n]
    memo[n]=fib(n-1)+fib(n-2)
    return memo[n]
```

 ZADANIE

Wersja rekurencyjna, która zwracałaby listę wyrazów od pierwszego do n-tego, jest problematyczna, tzn. nie może wyglądać tak jak klasyczne wywołanie z $\text{fib}(n-1) + \text{fib}(n-2)$, tylko

```
def fib(n, l=None):
    if n == 0: return [0]
    if n == 1: return [0, 1]
    if l is None: l = [0, 1]
    fib(n - 1, l)
    l.append(l[-1] + l[-2])
    return l
```

wersja kompaktowa

```
def fib(n, l = []):
    l += [0] if n==0 else [0,1] if n==1 else [(fib(n-1, l), l[-1]+l[-2])[1]]
    return l
```

Ta wersja nie robi „klasycznego” Fibonacciego ($\text{fib}(n-1) + \text{fib}(n-2)$), tylko tak naprawdę iterację ukrytą w rekurencji: dla $n \geq 2$ wywołuje tylko $\text{fib}(n-1, l)$, a nie dwa wywołania. Po powrocie dopisuje jeden nowy element $l[-1] + l[-2]$. Lista l jest ta sama w całym łańcuchu wywołań, więc nic nie kopiuje ani nie liczy ponownie. Złożoność czasowa i pamięciowa to $O(n)$.

Ograniczeniem rekurencyjnego podejścia jest głębokość stosu $\sim n$, więc praktycznie n musi być $< \sim 1000$ albo trzeba podnieść `recursionlimit` – odczytanie `sys.getrecursionlimit()`, ustawienie `sys.setrecursionlimit(...)`. Domyślnie Python ustawia limit głębokości rekurencji na 1000. Dla dużego n dostaniemy `RecursionError`.

Na temat argumentów funkcji

 ZADANIE

Oprócz parametrów formalnych, funkcja może otrzymać też argument w postaci `*name` (odpowiada to krotce argumentów pozycyjnych, poza tymi wyspecyfikowanymi bezpośrednio) oraz `**name` (odpowiada słownikowi ze wszystkimi kluczami argumentów, poza tymi formalnymi). Napisać funkcję drukującą argumenty:

```
def fun(par1, *par, **keypar):
    # tutaj kod
# przykład:
fun(111,"aaa",222,"ccc",klucz1="abc",klucz2=3.14,klucz3=True)
```

```
pierwszy: 111
*par: aaa 222 ccc
**keypar: klucz1 : abc  klucz2 : 3.14  klucz3 : True
```

Za pomocą specjalnych (opcjonalnych) parametrów / oraz * można wymusić podział argumentów na grupy argumentów tylko pozycyjnych, pozycyjnych lub z kluczem, specyfikowanych tylko kluczem.

```
def f(pos1, pos2, /, pos or kwd, *, kwd1, kwd2):
    -----
    |           |           |
    |           | Positional or keyword |
    |           |           |
    |           |           | - Keyword only
    |           |           |
    -- Positional only
```

- / - „zamyka” parametry pozycyjne, które nie mogą być nazwane. Przydaje np. w funkcjach wbudowanych (np. `math.pow(x, y)`), by nie trzeba było znać nazw argumentów.
- * - „otwiera” część keyword-only: wszystko po nim musi być nazwane

Dekoratory

ZADANIE

Napisz dekorator mydec (z wykorzystaniem biblioteki decorator), który:

- Przed wywołaniem dekorowanej funkcji wypisze linijkę: tekst nagłówek wyśrodkowany w polu szerokości 30 i wypełniony znakiem '-'.
- Przykład: `print('naglowek'.center(30, '-'))`
- Następnie wywoła dekorowaną funkcję z przekazanymi argumentami.
- Po zakończeniu wypisze linijkę stopki: tekst stopka wyśrodkowany w polu 30 i wypełniony znakiem '='.

```
@mydec
def bar(n):
    print(n)
```

```
>>> bar("lorem ipsum dolor sit amet")
```

Efekt:

```
----- naglowek -----
lorem ipsum dolor sit amet
===== stopka =====
```

Wymagane zainstalowanie:
pip install decorator

Użyj wzorca z `@decorator.decorator` i sygnatury:

```
@decorator.decorator
def mydec(func, *args, **kwargs):
    ...
    return func(*args, **kwargs)
```

 ZADANIE

Napisz dekorator mydec (z wykorzystaniem biblioteki decorator)...

Jeśli jest zainstalowane kilka wersji pythona, trzeba **sprawdzić**, gdzie trafi **instalowany** moduł. Na przykład:

```
>>> py -0p # zainstalowane wersje
-V:3.14t *   C:\Users\wit\AppData\Local\Programs\Python\Python314\python3.14t.exe
-V:3.14      C:\Users\wit\AppData\Local\Programs\Python\Python314\python.exe
-V:3.13t     C:\Users\wit\AppData\Local\Programs\Python\Python313\python3.13t.exe
-V:3.13      C:\Users\wit\AppData\Local\Programs\Python\Python313\python.exe

>>> py -version
pip 25.3 from C:\Users\wit\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)
# to oznacza, że pip install decorator zostanie dodany do python 3.13

>>> py -m pip -version # poniżej wskaże, gdzie trafi jeśli użyć launcher py
pip 25.2 from C:\Users\wit\AppData\Local\Programs\Python\Python314\Lib\site-packages\pip (python 3.14)

>>> py -m pip install decorator # taka instalacja trafi do wersji 3.14
# polecane albo jawne wskazanie docelowej wersji:

>>> py -3.13 -m pip install <paket> # na pewno do 3.13
>>> py -3.14 -m pip install <paket> # na pewno do 3.14
# albo tworzenie wirtualnego środowiska venv
```

Ciekawostka (raz jeszcze fib):

```
from functools import lru_cache
```

```
@lru_cache(None) # lub @cache w Py≥3.9
```

```
def fib(n):
```

```
    return n if n <= 1 else fib(n-1) + fib(n-2)
```

functools.lru_cache to **dekorator pamięci podręcznej** (cache) z polityką *Least Recently Used*. Zapamiętuje wyniki wywołań funkcji dla danych argumentów, żeby przy kolejnych identycznych wywołaniach **nie liczyć ponownie** – tylko zwrócić wynik z cache.

```
import decorator # lub: from decorator import decorator
@decorator.decorator # lub: @decorator
def mydec(func, arg, *args, **kwargs):
    print(' naglowek '.center(30, '-'))
    func(arg, *args, **kwargs)
    print(' stopka '.center(30, '='))

txt = "lorem ipsum dolor sit amet"

@mydec
def bar(n):
    print(n)

bar(txt)
```